

Solving the park-and-loop routing problem by branch-price-and-cut

Nicolás Cabrera, Jean-François Cordeau, Jorge E. Mendoza *

HEC Montréal, 3000 chemin de la Côte-Sainte-Catherine, Montréal, H3T 2A7, Canada

ARTICLE INFO

Keywords:

Vehicle routing problem
Branch-price-and-cut
Column generation
Transportation
Park-and-loop

ABSTRACT

The park-and-loop routing problem is a variation of the vehicle routing problem in which routes include a main tour that is completed using a vehicle and subtours that are carried out on foot after parking the vehicle. Additionally, the route duration and total walking distance are bounded. To solve the problem, we propose an exact solution method based on the branch-price-and-cut framework. In particular, our method uses problem-specific components to solve the pricing problem. We report on computational experiments carried out on a standard set of 40 instances with up to 50 customers. The results show that our method delivers solutions that compare favorably to existing metaheuristic algorithms, matching all previously best-known solutions and improving 11 of them in reasonable computational times. Moreover, our method provides optimality certificates for 39 out of the 40 instances.

1. Introduction

Let $\mathcal{G} = (\mathcal{V}, \mathcal{A})$ be a directed multigraph where $\mathcal{V}$ is the set of vertices and $\mathcal{A}$ denotes the set of directed arcs. The set of vertices comprises a depot 0 and the set of customers $C = \{1, \dots, C\}$. Each customer $i \in C$ has a service time s_i . There are k homogeneous workers available to serve the customers. These workers can either drive or walk between locations. Accordingly, the set of arcs $\mathcal{A} = \{(i, j)^m | m = \{d, w\}\}$ contains two arcs between each pair of vertices, namely, a driving arc (d) and a walking arc (w). Each arc $(i, j)^m \in \mathcal{A}$ has two main attributes: the distance δ_{ij} and the time η_{ij} . Each worker has a maximum daily walking distance ζ and is hired for a full working day lasting ϕ units of time. The maximum distance that a worker may walk between two points is θ . The fixed cost of hiring a worker is c^f . Additionally, there is a variable cost associated with driving (c^v per unit of distance). The park-and-loop routing problem (PLRP) consists in finding a set of least cost routes (starting and ending at the depot) while ensuring that: each customer is served precisely once; the total duration of each route does not exceed the working day duration; and the distance walked by any worker does not exceed the limit. Because it is a generalization of the well-known vehicle routing problem (VRP), the PLRP is NP-hard.

In order to serve the set of customers three types of routes can be designed: pure vehicle routes performed by a worker driving between customers; pure walking routes performed by a worker walking between customers; and finally park-and-loop routes that are vehicle routes with walking subtours. The number of customers in a walking subtour is not constrained. Fig. 1 shows a feasible solution to a small PLRP instance with 7 customers. In this solution, customers 1 and 2 are served by a worker driving a vehicle (i.e., a pure vehicle route), while customers 3 to 7 are served by a worker following a park-and-loop route. More specifically, the worker leaves (i.e., parks) the vehicle at the depot and walks to serve customer 3. The worker then walks back to the depot to pick up the vehicle and drives to customer 4. After serving customer 4, the worker parks the vehicle and walks to serve customers 5 and 7. The worker then walks back to customer 4. Finally, the worker drives to customer 6 and then returns to the depot.

* Corresponding author.

E-mail address: jorge.mendoza@hec.ca (J.E. Mendoza).

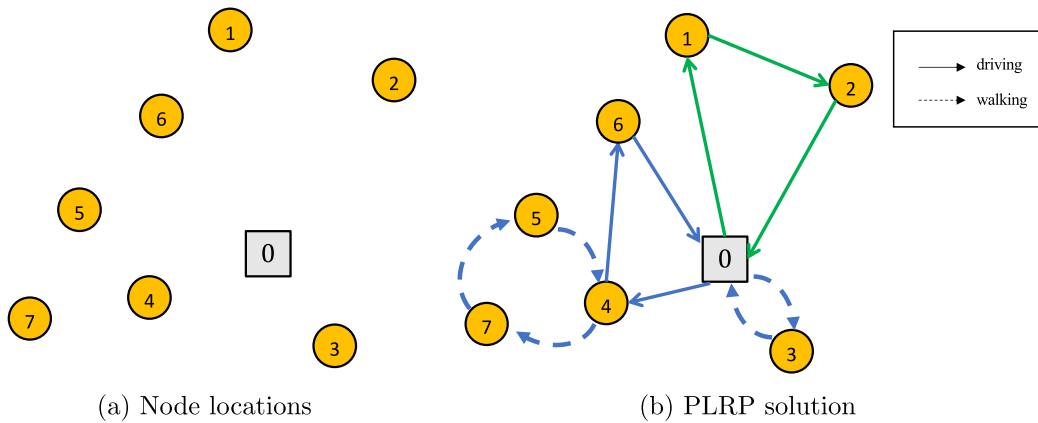


Fig. 1. PLRP instance and solution example.

The PLRP is closely related to the two-echelon last-mile delivery problem (2E-LMDP) discussed by [Martinez-Sykora et al. \(2020\)](#); a variant of the traveling salesman problem (TSP) consisting in finding a single park-and-loop route to serve a set of customers. The authors propose an exact branch-and-cut algorithm capable of solving instances with up to 30 customers. As opposed to the PLRP, the number of walking subtours starting at a given parking spot is limited to one. More recently, [Reed et al. \(2022\)](#) introduced the capacitated delivery problem with parking (CDPP). This problem extends the 2E-LMDP by allowing the worker to perform an unlimited number of walking subtours starting at the same parking spot. The authors propose a mixed integer programming (MIP) formulation that can solve instances with up to 50 customers. In addition, they describe a two-step heuristic that can handle instances with up to 100 customers. However, the authors limit the number of customers in a walking subtour to three. As stated previously, in the PLRP, the number of customers in a walking subtour is not constrained.

Another related problem is the two-echelon vehicle routing problem with time windows and mobile satellites (2EVRP-TM) introduced by [Li et al. \(2020\)](#). In this variant of the VRP each vehicle (i.e., truck) carries a fixed number of drones. These drones are launched either at the depot or at customer locations after parking the vehicle. After all the dispatched drones return to the vehicle, the route continues. To solve this problem, the authors propose an adaptive large neighborhood search (ALNS) heuristic that can handle instances with up to 100 customers. More recently, [Zhou et al. \(2023\)](#) presented a branch-and-price algorithm for solving a problem similar to the 2EVRP-TM in which the number of drones in each vehicle is treated as a decision variable. In this problem variant, once a drone returns to the truck, it may immediately depart to deliver a parcel to another customer. The authors test their algorithm on instances with up to 25 customers. As opposed to the PLRP, in the 2EVRP-TM it is possible that two (or more) subtours are carried out at the same time (by different drones) departing from the same parking spot. Another stream of research has focused on variants of the vehicle routing problem with drones (VRPD). In this setting, drones may be launched at one location and retrieved at another. This drastically changes the structure of the routes, which no longer follow a regular park-and-loop structure, and introduces synchronization issues that are not present in the PLRP. The interested reader is referred to [Tamke and Buscher \(2021\)](#) and [Yin et al. \(2023\)](#) for more details.

The PLRP is also related to the truck and trailer routing problem (TTRP) introduced by [Semet \(1995\)](#). This variant of the VRP considers a fleet of trucks pulling trailers to serve a set of customers. The problem also considers a set of decoupling locations (i.e., parking places), where trailers can be detached as some of the customers are only accessible by the truck without the trailer. Most of the work on the TTRP has focused on heuristic algorithms ([Chao, 2002](#); [Sheuerer, 2006](#); [Lin et al., 2009](#); [Villegas et al., 2011](#); [Derigs et al., 2013](#); [Villegas et al., 2013](#)). These methods are capable of providing high quality solutions for instances with up to 150 customers. In the TTRP, routes are constrained by the combined capacity of the truck and the trailer. In contrast, routes in the PLRP are constrained by the maximum walking distance and the working day duration.

In the last decade, researchers have turned their attention to developing exact methods for more restricted TTRP variants ([Belenguer et al., 2016](#); [Parragh and Cordeau, 2017](#); [Rothenbächer et al., 2018](#)). [Belenguer et al. \(2016\)](#) propose a branch-and-cut algorithm for the single truck and trailer routing problem with satellite depots (STTRPSD). This algorithm is capable of optimally solving instances with up to 50 customers when limiting the number of parking places to 10. [Parragh and Cordeau \(2017\)](#) propose a branch-and-price algorithm to solve the truck and trailer routing problem with time windows (TTRPTW). Their method is capable of solving instances with up to 100 customers. [Rothenbächer et al. \(2018\)](#) propose a branch-price-and-cut algorithm that outperforms the algorithm of [Parragh and Cordeau \(2017\)](#) on the same TTRPTW variant. Their method is capable of finding the optimal solution for 35 additional instances. As opposed to the TTRPTW, customers in the PLRP do not have an associated time window. Therefore, the latter can be expected to be more difficult to solve using branch-price-and-cut.

Another related problem is the doubly open park-and-loop routing problem (DOPLRP) introduced by [Cabrera et al. \(2022\)](#). This variant of the VRP consists in finding a set of least-cost routes that may start and end at any customer. To solve the DOPLRP, the authors propose a two-phase matheuristic called MSH. This approach was capable of handling instances with up to 3800 customers. As opposed to the PLRP, the time duration of each walking subtour is bounded. More recently, [Le Colleter et al. \(2023\)](#)

defined the park-and-loop routing problem with parking selection (PLRP-PS). The main difference with other problem variants is that the vehicle can only be parked at dedicated parking locations. To solve the PLRP-PS, the authors introduce a small and large neighborhood search metaheuristic (SLNS). The authors use new specific techniques to select parking spots that significantly speed up the algorithm. Their algorithm was capable of providing solutions for instances with up to 400 customers and 352 dedicated parking spots.

The closest problem to the PLRP is the VRP with transportable resources (VRPTR) introduced by Coindreau et al. (2019). In the VRPTR, a set of workers has to serve a set of customers. The workers can either walk or drive to their next location and are allowed to carpool (i.e., share a vehicle). To solve their VRPTR, the authors use a mixed integer linear program (MILP). Their experiments show that their MILP can only solve instances with up to 18 customers. Thus, they also propose a variable neighborhood search (VNS) algorithm. This method can solve instances with up to 50 customers with a running time limit of 10 h. The authors also studied a version of their problem in which carpooling is not allowed. The latter perfectly matches the PLRP definition. Le Colleter et al. (2023) reported results benchmarking VNS, MSH, and SLNS on the Coindreau et al. (2019) instances. Their study sets SLNS as the state-of-the-art algorithm since it unveiled eight new best-known solutions. Note, however, that neither of these methods is exact. Moreover, none of those approaches has been assessed with respect to a lower bound. In other words, no optimality gaps have been reported for the solutions they provide.

The contribution of this article is two-fold. From a methodological perspective, we propose a branch-price-and-cut algorithm to solve the PLRP. The key algorithmic component of our method is the pulse algorithm used to solve the pricing problem. The latter extends the procedure introduced in Lozano et al. (2016) to handle the park-and-loop structure of the routes and the inclusion of the subset row inequalities proposed by Jepsen et al. (2008). In addition, we present a set of acceleration strategies tailored to the PLRP. From a computational perspective, we perform extensive experiments on the set of 40 instances introduced by Coindreau et al. (2019), arguably the most widely used testbed for VRPs with park-and-loop structure. Our algorithm is the first to prove optimality for 39 of the instances. To further assess the advantages and limitations of our approach, we also perform experiments on a new challenging set of instances. In addition, we developed an online web application¹ that allows researchers to visualize and download the best-known solutions for the PLRP. They can also upload their solutions for plotting and checking.

This paper is organized as follows. Section 2 presents the mathematical formulation. Section 3 describes the proposed branch-price-and-cut algorithm. Section 4 presents the acceleration strategies that crucially improve the algorithm's performance. Section 5 contains the computational experiments. Finally, Section 6 presents the conclusions and outlines potential paths for future research.

2. Problem formulation

We define a route r as an ordered set of directed arcs starting and ending at the depot. The customers served in the route are represented by the set C_r . Let the subsets $\mathcal{A}_r^d$ and $\mathcal{A}_r^w$ contain the driving and the walking arcs in r respectively. Every arc $(i, j) \in \mathcal{A}_r^w$ must satisfy the condition $\delta_{ij} \leq \theta$. A route r is time-feasible if

$$\sum_{(i,j) \in \mathcal{A}_r^d \cup \mathcal{A}_r^w} \eta_{ij} + \sum_{i \in C_r} s_i \leq \phi. \quad (1)$$

Similarly, a route r is walking-feasible if

$$\sum_{(i,j) \in \mathcal{A}_r^w} \delta_{ij} \leq \zeta. \quad (2)$$

The cost c_r of a route is equal to the sum of the variable and the fixed cost, that is,

$$c_r = \sum_{(i,j) \in \mathcal{A}_r^d} \delta_{ij} c^v + c^f. \quad (3)$$

Let $\mathcal{R}$ be the set of all feasible routes and let a_{ir} be a parameter that takes the value 1 if and only if route $r \in \mathcal{R}$ serves customer $i \in C$. Finally, let x_r be a binary variable equal to 1 if route $r \in \mathcal{R}$ is selected and 0 otherwise. A set covering (SC) formulation for the PLRP can be stated as follows:

$$\min \sum_{r \in \mathcal{R}} x_r c_r \quad (4)$$

subject to

$$\sum_{r \in \mathcal{R}} a_{ir} x_r = 1 \quad \forall i \in C \quad (5)$$

$$\sum_{r \in \mathcal{R}} x_r \geq \left\lceil \frac{\sum_{i \in C} s_i}{\phi} \right\rceil \quad (6)$$

$$\sum_{r \in \mathcal{R}} x_r \leq k \quad (7)$$

$$x_r \in \{0, 1\} \quad \forall r \in \mathcal{R}. \quad (8)$$

¹ Available at <https://chairelogistique.hec.ca/en/scientific-data/>.

The objective function (4) minimizes the total cost. Constraints (5) ensure that all customers are served exactly once. Constraints (6) and (7) provide, respectively, a lower and an upper bound on the number of routes used to serve the set of customers. Constraints (8) are the domain restrictions. Note that the number of feasible routes $|\mathcal{R}|$ grows exponentially. Thus, solving SC by enumerating all the feasible routes is usually not possible. As an alternative, the SC can be solved using a branch-price-and-cut algorithm which is described next.

3. Solution method

In this section, we present an exact branch-price-and-cut algorithm (BPC) to solve the SC model. A BPC algorithm is a branch-and-bound algorithm in which, at each node of the enumeration tree, the linear relaxation of an integer formulation is solved using column generation (CG) and tightened by adding valid inequalities (i.e., cuts). For completeness, Section 3.1 describes the column generation algorithm. Section 3.2 defines the pricing problem. Section 3.3 describes the key strategies used in the pricing problem algorithm. Section 3.4 presents the valid inequalities. Finally, Section 3.5 describes the branching rules.

3.1. Column generation

CG is a solution method that can solve integer programs with a large number of decision variables (i.e., columns). To achieve this goal, two optimization problems are solved iteratively: A *master problem* that considers only a small subset of variables and a *pricing problem* that generates promising variables to be added to the master problem. If, at a given iteration, no variables are added to the master problem, the CG algorithm ends. We refer the interested reader to the book by Desaulniers et al. (2005) and the study by Lübbecke and Desrosiers (2005) for a review of techniques and applications of column generation.

In our case, the master problem corresponds to the relaxed version of formulation (4)–(8). The relaxed set covering problem (RSCP) considers only a subset of feasible routes (columns) $\bar{\mathcal{R}} \subseteq \mathcal{R}$ and is obtained by relaxing the integrality constraints on the x_r variables. Let $\pi_i \in \mathbb{R}$, $\sigma \geq 0$, and $\rho \leq 0$ be the dual variables associated with constraints (5), (6), and (7) respectively. The reduced cost of variable x_r is then given by the following expression:

$$\tau_r = c_r - \sum_{i \in C} a_{ir} \pi_i - \sigma - \rho. \quad (9)$$

Having $\min_{r \in \bar{\mathcal{R}}} \{\tau_r\} \geq 0$ ensures that the RSCP is solved optimally. If there exists a route r with $\tau_r < 0$, we add the corresponding variable x_r to the subset $\bar{\mathcal{R}}$. Therefore, the goal after solving the RSCP is to identify a route with negative reduced cost. This problem is referred to as the pricing problem and is the topic of the next section.

3.2. Pricing problem

Let $\mathcal{G}' = (\mathcal{V}', \mathcal{A}')$ be a directed multigraph, henceforth referred to as the *modified network*, where $\mathcal{V}'$ is the set of nodes including a start depot $\underline{0}$ and an end depot $\bar{0}$. Thus, $\mathcal{V}' = C \cup \{\underline{0}, \bar{0}\}$ and $\mathcal{A}' = \mathcal{A}^1 \cup \mathcal{A}^2 \cup \mathcal{A}^3 \cup \mathcal{A}^4 \cup \mathcal{A}^5$, where

- $\mathcal{A}^1 = \{(i, j)^d \mid i \in C \cup \{\underline{0}\}, j \in C\}$ is the set of driving arcs arriving to any customer,
- $\mathcal{A}^2 = \{(i, j)^w \mid i \in C \cup \{\underline{0}\}, j \in C \mid \delta_{ij} \leq \theta\}$ is the set of walking arcs arriving to any customer,
- $\mathcal{A}^3 = \{(i, \bar{0})^d \mid i \in C\}$ is the set of driving arcs going from any customer to the end depot,
- $\mathcal{A}^4 = \{(i, \underline{0})^w \mid i \in C \mid \delta_{i\underline{0}} \leq \theta\}$ is the set of walking arcs going from any customer to the start depot, and
- $\mathcal{A}^5 = \{(\underline{0}, \bar{0})^w\}$ is a fictitious arc connecting the start and end depots.

The attributes of the arc connecting $\underline{0}$ and $\bar{0}$ are set to zero. Note that arcs $\mathcal{A}^4$ are necessary in order to allow workers to start walking subtours from the depot. Fig. 2 shows how the routes in Fig. 1(b) are mapped to the modified network.

As mentioned previously, the objective of the pricing problem is to find routes from $\underline{0}$ to $\bar{0}$ with a negative reduced cost. With this in mind, we must define the reduced cost of each arc in $\mathcal{G}'$. The reduced cost of an arc $(i, j)^m \in \mathcal{A}'$ is defined as:

$$r_{ij} = \begin{cases} \delta_{ij} c^v - \pi_j, & (i, j)^m \in \mathcal{A}^1; \\ -\pi_j, & (i, j)^m \in \mathcal{A}^2; \\ \delta_{ij} c^v, & (i, j)^m \in \mathcal{A}^3; \\ 0, & (i, j)^m \in \mathcal{A}^4; \\ 0, & (i, j)^m \in \mathcal{A}^5. \end{cases} \quad (10)$$

In terms of network flows, the start depot offers one unit of flow that is demanded by the end depot. Thus, the formulation of the pricing problem partially follows that of the shortest path problem in which the weight of the arcs corresponds to their reduced cost. Nevertheless, in the context of the PLRP, a node in graph $\mathcal{G}'$ can only be visited once (unless the node is used as a parking spot), which means that paths are pseudo-elementary. Moreover, paths (i.e., routes) are constrained by two resources: the time ϕ and the walking distance ζ . Hence, the pricing problem corresponds to solving a variant of the elementary shortest path problem

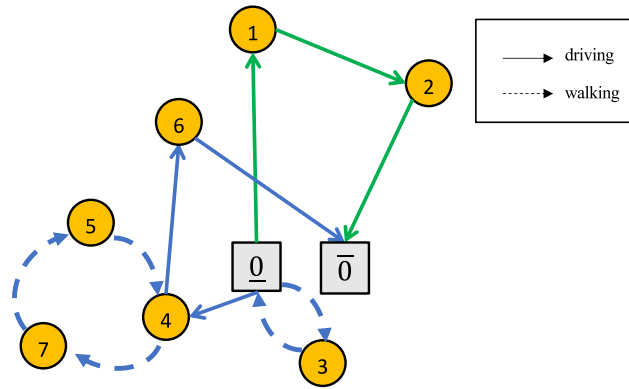


Fig. 2. Route representation on the modified network.

with resource constraints (ESPPRC) in which routes have a park-and-loop structure. We will refer to our specific pricing problem as the elementary shortest path problem with resource constraints and park-and-loop (ESPPRC-PL).

The ESPPRC itself is NP-hard. For this reason, considerable research effort has been devoted to designing algorithms to solve versions of the problem in which the elementarity constraints are partially (or completely) relaxed. Even though the quality of the bounds provided by the MP may be significantly reduced, solving these relaxations is usually faster. The interested reader is referred to [Costa et al. \(2019\)](#) for a detailed review of ESPPRC relaxations. The resulting shortest path problem with resource constraints (SPPRC) is often solved using dynamic programming algorithms. To put it simply, these algorithms are based on the idea of extending labels following a breadth-first search paradigm. Each label represents a partial path and stores the consumption of resources. In order to avoid a complete enumeration of the partial paths in the network, labels are discarded using problem-specific dominance rules. These algorithms can be very fast for small and medium-sized networks. Nevertheless, the number of labels to store can be extremely large. Thus, they may fail to scale to large networks.

Another stream of research has focused on solving the ESPPRC without explicitly applying any relaxations. As a result, the bounds provided by the MP are typically tighter (stronger), which potentially leads to the exploration of smaller branch-and-price trees. One of the algorithms that is commonly used in this setting is the pulse algorithm (PA) proposed by [Lozano et al. \(2016\)](#). The PA is based on a recursive depth-first search that crucially combines multiple pruning strategies that discard partial paths. A vital feature of the PA is that it does not rely on asserting dominance between labels. More crucially, labels are never stored at nodes, which significantly decreases the memory requirements of the algorithm. Another interesting feature of the PA is its flexibility. In recent years, it has been adapted (with mild changes) to solve the orienteering problem with time windows ([Duque et al., 2014](#)), the bi-objective shortest path problem ([Duque et al., 2015](#)), the weight-constrained shortest path problem with replenishment ([Bolívar et al., 2014](#)), the shortest α -reliable path problem ([Corredor-Montenegro et al., 2021](#)), and, more recently, the least expected travel-time path problem ([Yamín et al., 2022](#)). The PA has been also used as a component in algorithms to solve other hard combinatorial optimization problems ([Restrepo et al., 2012](#); [Montoya et al., 2016](#); [Lozano and Smith, 2017](#); [Arslan et al., 2018](#); [Schrotenboer et al., 2019](#)).

Based on the above observations, we decided to adapt the PA to solve the ESPPRC-PL. Following the same intuition of the original PA, our algorithm propagates pulses through the network from a start node (i.e., the start depot) $\underline{0}$ to an end node (i.e., the end depot) $\bar{0}$. While traversing the network node by node, the pulse builds a partial path $\mathcal{P}$ that includes all the nodes already visited. Additionally, the pulse contains information on the attributes associated with the path, such as the cumulative reduced cost or the resource consumption. Whenever a pulse reaches the end node $\bar{0}$ it contains all the information of a feasible path $\mathcal{P}$ from $\underline{0}$ to $\bar{0}$. If the path has a negative reduced cost, it can be added to the subset $\bar{\mathcal{R}}$ and the best solution can be updated.

The PA ensures that the optimal path $\mathcal{P}^*$ is always found by implicitly enumerating all paths from $\underline{0}$ to $\bar{0}$. However, one can easily truncate the PA to accelerate the search by solving the problem heuristically, as it is discussed in Section 4.2. To prevent the PA from explicitly enumerating all possible paths, the algorithm uses a set of pruning strategies. These strategies allow the PA to stop (prune) the propagation of a partial path as soon as there is enough evidence that the partial path will not improve the current best solution or that it will lead to an infeasible solution. Note that stopping a partial path from propagating allows for discarding a large number of paths, as it discards all the paths that begin with it. Thus, the earlier a partial path is stopped, the better. This idea is shared with other algorithms, like branch-and-bound, where an implicit enumeration is performed. Similarly, the strength of the pulse algorithm depends on the pruning strategies. In what follows the terms *stopping* or *pruning* a path are used interchangeably.

Algorithm 1 presents the main logic of the pulse algorithm. Line 1 initializes the partial path. Lines 2 to 4 initialize the reduced cost and the cumulative resource consumption. Line 5 runs the bounding procedure given the bound step size Δ and the bounding time limits $[\bar{t}, \underline{t}]$. Line 6 extends a pulse at the start node. Finally, line 7 returns the optimal path. Note that in the case where reaching the end node $\bar{0}$ is not possible due to the resource constraints (i.e., the pricing problem is infeasible), the optimal path ends up empty.

Algorithm 1 pulseSearch function

Require: $\mathcal{G}'$, directed multi graph; ϕ , duration limit; ζ , walking distance limit; $\underline{0}$, start node; $\bar{0}$, end node; Δ , bound step size; $[\bar{t}, \underline{t}]$, bounding time limits.
Ensure: $\mathcal{P}^*$, optimal path.

```

1:  $\mathcal{P}^* \leftarrow \emptyset$ 
2:  $r(\mathcal{P}) \leftarrow 0$ 
3:  $t(\mathcal{P}) \leftarrow 0$ 
4:  $w(\mathcal{P}) \leftarrow 0$ 
5:  $\text{bound}(\mathcal{G}', \Delta, [\bar{t}, \underline{t}])$  ▷ see Section 3.3.2
6:  $\text{pulse}(\underline{0}, r(\mathcal{P}), t(\mathcal{P}), w(\mathcal{P}), \mathcal{P})$  ▷ see Algorithm 2
7: return  $\mathcal{P}^*$ 

```

Algorithm 2 shows the recursive procedure `pulse`, where $\Gamma_w^+(i)$ corresponds to the set of arcs leaving node i on foot and $\Gamma_d^+(i)$ by driving. Lines 1 to 4 use the pruning strategies, namely, infeasibility, bounds, rollback, and path completion, to try to prune a partial path. If the pulse is not pruned, line 5 adds the node to the partial path. From lines 6 to 10, the algorithm recursively propagates the pulse by driving to node j . From lines 11 to 17, the algorithm recursively propagates the pulse using Algorithm 3 by walking to node j , thus parking the vehicle at node i .

Algorithm 2 pulse function

Require: i , current node; r , cumulative reduced cost; t , cumulative time; w , cumulative walking distance; $\mathcal{P}$, partial path.

```

1: if  $\neg \text{feasibility}(i, t(\mathcal{P}), w(\mathcal{P}))$  ▷ see Section 3.3.1
2:   if  $\neg \text{bounds}(i, r(\mathcal{P}), t(\mathcal{P}))$  ▷ see Section 3.3.2
3:     if  $\neg \text{rollback}(i, r(\mathcal{P}), t(\mathcal{P}), w(\mathcal{P}), \mathcal{P})$  ▷ see Section 3.3.3
4:       if  $\neg \text{complete\_path}(i, r(\mathcal{P}), t(\mathcal{P}), w(\mathcal{P}), \mathcal{P})$  ▷ see Section 3.3.4
5:          $\mathcal{P}' \leftarrow \mathcal{P} \cup \{i\}$ 
6:         for  $j \in \Gamma_d^+(i)$  do
7:            $r(\mathcal{P}') \leftarrow r(\mathcal{P}) + r_{ij}$ 
8:            $t(\mathcal{P}') \leftarrow t(\mathcal{P}) + \eta_{ij} + s_j$ 
9:            $\text{pulse}(j, r(\mathcal{P}'), t(\mathcal{P}'), w(\mathcal{P}), \mathcal{P}')$ 
10:        end for
11:        for  $j \in \Gamma_w^+(i)$  do
12:           $r(\mathcal{P}') \leftarrow r(\mathcal{P}) + r_{ij}$ 
13:           $t(\mathcal{P}') \leftarrow t(\mathcal{P}) + \eta_{ij} + s_j$ 
14:           $w(\mathcal{P}') \leftarrow w(\mathcal{P}) + \delta_{ij}$ 
15:           $ps \leftarrow i$ 
16:           $\text{pulse\_parked}(j, r(\mathcal{P}'), t(\mathcal{P}'), w(\mathcal{P}'), ps, \mathcal{P}')$ 
17:        end for
18:      end if
19:    end if
20:  end if
21: end if
22: return void

```

Algorithm 3 shows the recursive procedure `pulse_parked`. Lines 1 to 3 try to prune the partial path using the infeasibility, bounds, and rollback pruning strategies. If the pulse is not pruned, line 4 adds the node to the partial path. From lines 5 to 22, the algorithm recursively propagates the pulse by walking to node j . At line 9, we check if the path is returning to the parking spot. If so, at line 14 the algorithm checks if it is possible to prune the partial path by using the subtour fixing strategy. If the partial path is not pruned, the algorithm recursively propagates the pulse using Algorithm 2.

The following section provides further detail regarding the pruning strategies used by the PA.

3.3. Pruning strategies

In Sections 3.3.1, 3.3.2, and 3.3.3 we describe the adaptation of the original PA pruning strategies proposed by Lozano et al. (2016), namely, infeasibility, bounds, and rollback pruning. In Section 3.3.4 we present the path completion strategy (line 4 in Algorithm 2) adapted from Cabrera et al. (2020) which was used to solve the constrained shortest path problem (CSP). Finally, in Section 3.3.5 we describe a new pruning strategy for the PA specifically tailored for our pricing problem called subtour fixing (line 16 in Algorithm 3).

Algorithm 3 pulse_parked function

Require: i , current node; r , cumulative reduced cost; t , cumulative time; w , cumulative walking distance; ps , parking spot; $\mathcal{P}$, partial path.

```

1: if  $\neg \text{feasibility}(i, t(\mathcal{P}), w(\mathcal{P}))$  then ▷ see Section 3.3.1
2:   if  $\neg \text{bounds}(i, r(\mathcal{P}), t(\mathcal{P}))$  then ▷ see Section 3.3.2
3:     if  $\neg \text{rollback}(i, r(\mathcal{P}), t(\mathcal{P}), w(\mathcal{P}), \mathcal{P})$  then ▷ see Section 3.3.3
4:        $\mathcal{P}' \leftarrow \mathcal{P} \cup \{i\}$ 
5:       for  $j \in \Gamma_w^+(i)$  do
6:          $r(\mathcal{P}') \leftarrow r(\mathcal{P}) + r_{ij}$ 
7:          $t(\mathcal{P}') \leftarrow t(\mathcal{P}) + \eta_{ij} + s_j$ 
8:          $w(\mathcal{P}') \leftarrow w(\mathcal{P}) + \delta_{ij}$ 
9:         if  $ps = j$  then
10:           $t(\mathcal{P}') \leftarrow t(\mathcal{P}') - s_j$ 
11:          if  $j \in \mathcal{C}$  then
12:             $r(\mathcal{P}') \leftarrow r(\mathcal{P}) + \pi_j$ 
13:          end if
14:          if  $\neg \text{subtour\_fixing}(j, t(\mathcal{P}), \mathcal{P})$  then ▷ see Section 3.3.5
15:            pulse( $j, r(\mathcal{P}'), t(\mathcal{P}'), w(\mathcal{P}'), \mathcal{P}'$ )
16:          end if
17:        else if  $ps \neq j$  then
18:          pulse_parked( $j, r(\mathcal{P}'), t(\mathcal{P}'), w(\mathcal{P}'), ps, \mathcal{P}'$ )
19:        end if
20:      end for
21:    end if
22:  end if
23: end if
24: return void

```

3.3.1. Infeasibility pruning

The intuition of this pruning strategy is to stop a pulse as soon as it becomes evident that it will not be able to reach the end node $\bar{0}$ while meeting the resource constraints. Thus, we can safely stop a partial path $\mathcal{P}$ from propagating if any of the following conditions holds:

- $t(\mathcal{P}) > \phi$;
- $w(\mathcal{P}) > \zeta$.

Discarding infeasible partial paths is often used as a key strategy to improve the performance of labeling algorithms. Note that this strategy could easily be extended to include other route constraints such as the presence of time windows at customer locations or a time limit on each walking subtour.

3.3.2. Bounds pruning

Similar to the infeasibility pruning strategy, we can stop a pulse from propagating if there is enough information to prove that the current partial path will not lead to improving the best solution found so far. More specifically, if there is evidence that the partial path will not be able to decrease the best objective function $r(\mathcal{P}^*)$ we can stop the partial path from propagating. With this purpose, we use a bounding scheme that computes lower bounds $\underline{r}(i, t(\mathcal{P}))$ for every node $i \in \mathcal{G}'$ and for a set of possible values of time resource consumption $t(\mathcal{P})$. More specifically, these bounds contain the minimum reduced cost from any node i to the end node $\bar{0}$ given a partial resource consumption $t(\mathcal{P})$.

To compute the lower bounds, we solve an ESPPRC-PL from every node $i \in \mathcal{G}'$ to $\bar{0}$ given a time consumption of $t(\mathcal{P}) = \bar{t} - \Delta$. These problems are overly-constrained as the pulse only has Δ units of time available to reach the end node $\bar{0}$. Accordingly, the pulse algorithm can easily solve these problems to optimality. Each of the solutions found is a valid lower bound on the minimum reduced cost that can be obtained from node i given a time consumption $t(\mathcal{P}) \geq \bar{t} - \Delta$. After finding these bounds, we proceed to solve an ESPPRC-PL from every node $i \in \mathcal{G}'$ to $\bar{0}$ given a time consumption of $t(\mathcal{P}) = \bar{t} - 2\Delta$. Although the resulting problems are less constrained, we already have vital information for partial paths with a time consumption between $[\bar{t} - \Delta, \bar{t}]$. We continue with this procedure, solving ESPPRC-PL problems with $\{\bar{t} - 3\Delta, \bar{t} - 4\Delta, \dots, \underline{t}\}$. Note that at the end of this procedure, we have a lower bound for every node and every discrete time step between $\underline{t}$ and $\bar{t}$. With this information, we can prune a partial path $\mathcal{P}$ if $r(\mathcal{P}) + \underline{r}(i, t(\mathcal{P})) \geq r(\mathcal{P}^*)$.

For further details regarding this strategy, the reader is referred to [Lozano et al. \(2016\)](#). Note that in this section we describe the bounding scheme using only the time consumption $t(\mathcal{P})$. In preliminary experiments, we considered computing additional bounds with respect to the walking distance $w(\mathcal{P})$. However, this did not lead to a major improvement in performance.

3.3.3. Rollback pruning

The choice between a depth-first search (DFS) and a breadth-first search (BFS) strategy has been widely studied in the literature as it affects the performance of every labeling setting/correcting algorithm. Although it is possible to affect the behavior of the pulse algorithm to make a BFS exploration through the usage of *pulse queues* as presented in [Cabrera et al. \(2020\)](#), in this article we consider a version of the pulse algorithm that follows a pure DFS strategy.

In some cases, this behavior could lead to exploring vast unpromising regions of the search space, before backtracking to correct poor decisions made earlier. To overcome this problem, the rollback pruning strategy re-evaluates the last choice made. More specifically, consider a partial path $P_{0,i}$ from 0 to i that is extended to node l and then reaches node j , namely, $P_{0,j} = P_{0,i} \cup l \cup j$. Once the partial path $P_{0,j}$ reaches node j , we compare it with the partial path which skips node l , that is, $P'_{0,j} = P_{0,i} \cup j$. If $r(P'_{0,j}) \leq r(P_{0,j})$ and $t(P'_{0,j}) \leq t(P_{0,j})$ we can prune path $P_{0,j}$ as it is dominated by path $P'_{0,j}$. As pointed out by [Lozano et al. \(2016\)](#), this strategy is based on simple arithmetic calculations for $r(P'_{0,j})$ and $t(P'_{0,j})$ and does not rely on storing any kind of labels.

3.3.4. Path completion

One of the main drawbacks of the PA, as it was presented by [Lozano et al. \(2016\)](#), is that the best path P^* is only updated when the end node 0 is reached. Thus, the main purpose of the path completion strategy is to update the best path (primal bound) at intermediate nodes in the network.

To do so, we take advantage of the information computed in the bounding procedure presented in Section 3.3.2. Formally, let us consider a partial path $P_{0,i}$ arriving to node i . The path completion strategy adds the minimum reduced cost path $P_{i,0}^{r'}$ given a time consumption $t(P_{0,i})$ to the partial path $P_{0,i}$, that is, $P_{0,0} = P_{0,i} \cup P_{i,0}^{r'}$. If the completed path is feasible and the reduced cost is lower than the current primal bound, we update the incumbent solution accordingly. Furthermore, we can stop the incoming partial path $P_{0,i}$ from propagating, because (by construction) we know that the complete path is already the minimum reduced cost path stemming from this partial path given the current time consumption. This procedure is used in Line 4 of Algorithm 2 and is adapted from the path completion strategy proposed by [Cabrera et al. \(2020\)](#) for the constrained shortest path problem.

3.3.5. Subtour fixing

Consider a partial path $P_{0,j}$ from 0 to j in which j is used as a parking spot. Moreover, consider a walking subtour s visiting a subset C_s^j of three or more customers $C_s^j = \{i_1, i_2, \dots, i_{|C_s^j|}\}$ and stemming from node j . The number of possible walking subtours visiting all the customers in C_s^j and using j as a parking spot can be calculated as $|C_s^j|!$. Thus, during the recursive search, the algorithm can visit j to retrieve the vehicle following several different sequences in which all the customers in C_s^j are visited. The total reduced cost associated with each of these sequences is the same (i.e., $\sum_{i \in C_s^j} -\pi_i$), while the total time may differ. Note that a slower walking subtour may result in the impossibility of visiting other customers using the partial path $P_{0,j}$.

Once again, as the algorithm follows a pure DFS strategy, is possible that many partial paths will be explored before backtracking and correcting the sequence followed for visiting the customers in C_s^j . For this reason, every time a partial path completes a walking subtour s we check if it is possible to stop that path from propagating. In practice, we follow two steps. First, we check if it is the first time (since the BPC algorithm started) that the customers C_s^j are visited by a walking subtour stemming from j . If so, we solve a traveling salesman problem minimizing the total walking time and store the value $t^*(C_s^j)$ in memory. If not, we retrieve the value stored previously. Second, we compare the current time of the subtour $t(s)$ with the best time $t^*(C_s^j)$. If the total time $t(s)$ of the subtour s currently included in the partial path $P_{0,j}$ is larger than the total time of the optimal TSP (i.e., $t(s) > t^*(C_s^j)$) we can prune the path and thus avoid exploring paths that use an inefficient walking subtour.

3.4. Valid inequalities

The optimal solution of the RSCP can be fractional. In that case, before applying branching decisions we first try to improve (lift) the lower bound. To do so, we draw upon valid inequalities (cuts). Particularly, we include the subset row inequalities proposed by [Jepsen et al. \(2008\)](#) for subsets of three customers. These inequalities have been used in different applications as the multi-depot vehicle routing problem ([Contardo and Martinelli, 2014](#)), the vehicle routing problem with time windows ([Costa et al., 2019](#)), and the two-echelon capacitated vehicle routing problem ([Marques et al., 2020](#)). The subset row inequalities with $|S| = 3$ are defined as

$$\sum_{r \in R} \left\lfloor 1/2 \sum_{i \in S} a_{ir} \right\rfloor x_r \leq 1, \quad \forall S \subseteq C. \quad (11)$$

Each of these inequalities has an associated dual variable $\beta_S \leq 0$. These inequalities ensure that for a given subset $S \subseteq C$ the number of routes serving two or more customers is less or equal to 1.

To separate these inequalities we enumerate all customer triplets and check if the inequality is violated in the current optimal solution. It has been observed by multiple researchers that even if subset row inequalities tend to have a positive impact on the quality of the lower bound, they significantly increase the complexity of solving the pricing problem. Thus, in line with [Jepsen et al. \(2008\)](#), we allow our BPC algorithm to add up to φ at each iteration with a minimum violation of ε . Inequalities with greater violations are given priority. In our BPC, we set φ to 5 and ε to 0.1.

Note that adding these inequalities modifies the definition of the reduced cost of a route. More specifically, if we denote S as the subset of triplets of customers for which the subset row inequality has been generated and added to the master problem, then the reduced cost of a route is defined as

$$\tau_r = c_r - \sum_{i \in C} a_{ir} \pi_i - \sigma - \rho - \sum_{S \in \mathcal{S}} \beta_S \left\lfloor \frac{1}{2} \sum_{i \in S} a_{ir} \right\rfloor. \quad (12)$$

Adding a subset row inequality implies that a penalty of β_S must be paid if two or more customers of the corresponding triplet are served by the route. To account for this term using the PA, we add a new resource for each subset in $\mathcal{S}$ that stores the number of times that a customer in the subset has been visited. If, while extending a pulse this value reaches a value of 2 we subtract the β_S from the cumulative reduced cost. The reader should note that all the pruning strategies outlined in Section 3.3 can still be used without any changes. This is an advantage compared to algorithms that rely on assessing dominance between two partial paths (labels) in which extending the dominance criteria is required.

In addition, to strengthen the linear relaxation of the SC formulation, we lifted constraint (6) using the value of the objective function of the minimum spanning tree on the graph $\mathcal{G}$, as follows:

$$\left\lceil \frac{\sum_{i \in C} s_i + MST(\mathcal{G})}{\phi} \right\rceil \leq \sum_{r \in \mathcal{R}} x_r. \quad (13)$$

Constraint (13) ensures that the number of routes in the solution of the RSCP is at least the minimum number of routes needed to serve all the customers. Lifting this constraint does not have any impact on the pricing problem structure.

3.5. Branching rules

Adding the inequalities outlined in Section 3.4 does not guarantee that the optimal solution of the RSCP will be integral. In the case in which the optimal solution of the RSCP is still fractional, we resort to branching on the arc flow variables. To do so, we define a_{ijr}^m as the number of times an arc $(i, j)^m \in \mathcal{A}$ appears in route r . Then, for each arc $(i, j)^m \in \mathcal{A}$ it is possible to compute the number of times it appears in a solution as

$$b_{ij}^m = \sum_{r \in \mathcal{R}} a_{ijr}^m. \quad (14)$$

We select for branching the arc for which the value of b_{ij}^m is closest to 0.5. In the case of a tie, we prioritize driving arcs.

Branching on an arc implies creating two child nodes for the branch-and-price tree: one child in which the arc is forbidden (the value is set to zero) and one child in which the arc is fixed (the value is set to one). These conditions are enforced locally in the pricing problem by modifying the graph $\mathcal{G}'$. If an arc must be forbidden, the arc is simply removed from graph $\mathcal{G}'$. However, fixing an arc is not as straightforward as it depends on the transportation mode. If the arc (i, j) is a driving arc, we remove all the driving arcs starting from node i and ending at any node different than j . Moreover, we remove all the driving arcs ending at node j that start at any node different than i . In addition, we remove the walking arcs $(i, j)^w$ and $(j, i)^w$. If the arc (i, j) is a walking arc, we remove all the walking arcs starting from node i and ending at any node different than j . We also remove all the walking arcs ending at node j that start at any node different than i . In addition, we remove the driving arcs $(i, j)^d$ and $(j, i)^d$. In order to select the next node to branch on we use best bound search.

4. Acceleration strategies

We now describe several ideas that we use to speed up our BPC algorithm.

4.1. Dual stabilization

Usually, CG-based algorithms suffer from slow convergence, a phenomenon called the *tailing-off effect* (Desaulniers et al., 2005). An important technique for alleviating this issue is to implement a dual stabilization method. In this work, we implemented the α -schedule procedure presented by Pessoa et al. (2013). This procedure aims to correct the values of the dual variables used to solve the pricing problem based on previous dual solutions. Algorithm 4 shows the pseudocode of the procedure. Line 1 initializes the value of l . Line 2 initializes the value of the dual variables. Line 3 updates the smoothing factor. Line 4 computes the value of the dual variables that will be used for solving the pricing problem. Line 5 updates the value of l . Line 6 calls the pricing problem solved with the procedure described in Section 3.2. Solving the pricing problem with the modified dual variables may result in the impossibility to find a column with a negative reduced cost, even if such a column exists. This phenomenon is called *mis-pricing*. If *mis-pricing* occurs the algorithm returns to line 3. Otherwise, the number of iterations is updated, and the master problem is solved once again.

Algorithm 4 α -scheduling function**Require:** α , smoothing factor.

```

1:  $l \leftarrow 1$ 
2:  $\pi^0 \leftarrow \pi^{in}$ 
3:  $\bar{\alpha} \leftarrow [1 - l(1 - \alpha)]^+$ 
4:  $\pi^{sep} = \bar{\alpha}\pi^0 + (1 - \bar{\alpha})\pi^{out}$ 
5:  $l \leftarrow l + 1$ 
6: Call the pricing problem with  $\pi^{sep}$ 
7: if Mis-pricing occurs then
8:   Go to step 3
9: else
10:   $t \leftarrow t + 1$ 
11:  Solve the master problem
12:  Go to step 1
13: end if

```

4.2. Heuristic pulse algorithm

The pricing problem does not have to be solved to optimality at every iteration of the CG. Thus, it is a common practice to design heuristics to quickly find promising solutions in the first iterations of the CG (Desaulniers et al., 2005). In our case, we can easily truncate the PA to solve the pricing problem heuristically by imposing a stopping criterion. More specifically, we can heuristically stop the PA from propagating more pulses if the number of paths found with negative reduced cost reaches Y . Moreover, we can impose a time limit Λ . Then, if the CPU time for solving the pricing problem reaches Λ and the PA has already found a promising path, we stop the PA.

Furthermore, note that by allowing the PA to find paths with a park-and-loop structure, the complexity of solving the pricing problem increases heavily. However, it is possible that in some iterations of the CG, paths without subtours may have a negative reduced cost. Accordingly, we adopt a leveled pricing strategy in which we first run the PA without considering the walking arcs. Only if the algorithm was not able to find promising paths do we proceed to run the PA by allowing walking subtours.

4.3. Initialization

It is well known that the performance of a CG algorithm is affected by the initial set of columns. Usually, including a high-quality set of initial columns can help the algorithm perform a better estimation of the dual variables associated with the RSCP constraints. Although one could initialize the pool of columns using $|C|$ routes visiting one customer, in our implementation we use the output of the sampling phase of the MSH matheuristic proposed by Cabrera et al. (2022). For the sake of completeness, we briefly describe the procedure here.

Algorithm 5 presents the main logic of the sampling phase of MSH. Line 1 initializes the set of initial routes $\bar{R}$. Line 2 initializes the iteration number. From lines 4 to 13, the algorithm populates $\bar{R}$ using a set of TSP heuristics $\mathcal{H}$ and the splitting procedure $split(\cdot, \cdot)$. Line 4 randomly selects a TSP heuristic h from $\mathcal{H}$. Line 5 generates a giant route τ^t visiting all customers using h . Line 6 generates a solution, denoted as s^t . Line 7 joins the routes in solution s^t to set $\bar{R}$. Lines 8–12 update the incumbent solution. Line 15 returns the set of initial routes that will be used by our BPC.

The key algorithmic component of the sampling phase of MSH is the $split(\cdot, \cdot)$ procedure used to extract a solution s^t from the giant TSP-like tour in Line 6. The split procedure follows two steps. In the first step, it constructs a directed acyclic graph defined by a set of nodes $N = (v_0, v_1, \dots, v_i, \dots, v_n)$ and the set of arcs A . Node v_0 is a dummy node, while nodes numbered 1 to n represent the customer in the i -th position of the giant tour τ^t . Each arc $(i, j) \in A$ represents a feasible route $r(v_{i+1}, v_j)$ visiting customers $(v_{i+1}, \dots, v_j)$. To evaluate if an arc should be added to G (line 12) it solves a subproblem that can be seen as a multi-resource version of the single truck and trailer routing problem with satellite depots (STTRPSD) proposed in Villegas et al. (2010). The solution to this subproblem yields a route with one or more walking subtours. In the second step, the split procedure finds the shortest path from v_0 to v_n in G . The set of arcs (i.e., routes) along the shortest path corresponds to a feasible solution s^t .

In our BPC algorithm, we use the default parameters of the MSH. Namely, the number of iterations is set to 2500, and the time limit is set to 60 s. For further detail regarding the MSH, the reader is referred to Cabrera et al. (2022).

5. Computational experiments

In this section, we present the computational experiments that we performed on a set of standard instances from the literature. Our goal is to analyze the performance of the proposed BPC algorithm and its main components. In addition, we describe a web application that can be used to check the quality of the solutions found by any researcher working on the PLRP or related variants.

Algorithm 5 MSH sampling function**Require:** G , graph; $\mathcal{H}$, heuristic set; T , iteration limit; Q , time limit.**Ensure:** initial solution $\bar{\mathcal{R}}$

```

1:  $\bar{\mathcal{R}} \leftarrow \emptyset$ 
2:  $t \leftarrow 1$ 
3: while  $t < T \wedge \text{samplingTime} < Q$  do
4:    $h \leftarrow \mathcal{H}$ 
5:    $\tau^t \leftarrow h(G)$ 
6:    $s^t \leftarrow \text{split}(G, \tau^t)$ 
7:    $\bar{\mathcal{R}} \leftarrow \bar{\mathcal{R}} \cup s^t$ 
8:   if  $t = 1$  then
9:      $s^* \leftarrow s^t$ 
10:  else if  $f(s^t) < f(s^*)$  then
11:     $s^* \leftarrow s^t$ 
12:  end if
13:   $t \leftarrow t + 1$ 
14: end while
15: return  $\bar{\mathcal{R}}$ 

```

The BPC algorithm was implemented in Java using the jORLib² library and compiled using Java 1.8.0_331. The experiments were performed on an Intel core i7 @2.30 GHz Quad-Core processor with 12 GB of RAM. We used CPLEX 20.1 to solve the SC formulation and the RSCP. Due to the randomness induced by the initialization procedure described in Section 4.3, for every experiment, we ran five replicates. On each replicate, we used a different value from the set $\{1, 2, 3, 4, 5\}$ to seed Java's pseudo-random number generator. This way we ensure consistency across the initial solutions used by each algorithm and validate the consistency of the proposed BPC. We also set a time limit of 2 h for every run.

5.1. Test instances

To assess the efficiency and effectiveness of the BPC algorithm, we use the set of instances proposed by Coindreau et al. (2019) for the VRPTR without carpooling. Each instance considers a number of customers n in the set $\{20, 30, 40, 50\}$ located inside a square grid of 10 km by 10 km. The depot is located at the center of the grid. The distance between nodes (customers and depot) is the Euclidean distance. In addition, to compute driving and walking times they consider a driving speed of 30 km/h and a walking speed of 4 km/h. The customer service times range from 20 to 35 min. The maximum daily walking distance for each worker is 5 km and the day duration is 7 h (i.e., 420 min). It is worth recalling that any customer location can be used as a parking spot. For each instance, Coindreau et al. (2019) fixed the number of available workers as the number of routes in the corresponding VRP solution. The fixed cost is set to 0 \$ and the variable cost is set to 1 \$/km. The objective is to minimize the total cost. An instance is referred to as “n_A_i”, where n stands for the number of customers, A represents the size of the used time window (i.e., all day), and i denotes the instance unique identifier. A total of 10 instances are considered for each instance size (i.e., number of customers). After fine tuning, we set the bound step size in the PA (Δ) to 15. The bounding time limits $[l, \bar{l}]$ are set to $[120, 420]$. In addition, the maximum number of paths Y is set to 10 and the time limit Λ to 15 s. Finally, the parameter α used to stabilize the values of the dual variables is set to 0.8.

5.2. Assessing the BPC performance

In this section, we analyze the performance of the proposed BPC and compare it to that of the state-of-the-art algorithms for the PLRP, namely, the VNS introduced by Coindreau et al. (2019), the MSH designed by Cabrera et al. (2022), and the SLNS developed by Le Colleter et al. (2023).

Table 1 compares the performance of the proposed branch-price-and-cut algorithm in the best performing replicate against the benchmark algorithms (metaheuristics) in terms of solution quality. Each row corresponds to an instance size. Columns 2, 5, 8, and 11 show the number of best-known solutions (BKSS) found by each algorithm. Columns 3, 6, 9, and 12 report the average gap with the best-known solution. Columns 4, 7, 10, and 13 show the maximum gap with the best-known solution.

As the results show, BPC matched all previous best-known solutions and unveiled 11 new best-known solutions in all the replicates. On the subset of instances with 20 customers, both MSH and SLNS matched all the best-known solutions. However, as the number of customers increases, the quality of the solutions found decreases.

Table 2 compares the performance of BPC in the best performing replicate against the benchmark algorithms in terms of the optimality gap. To compute the optimality gap we used the lower bound found by our BPC. Similar to Table 1 the results are grouped by instance size. Columns 2, 5, 8, and 11 show the number of optimal solutions found by each algorithm. Columns 3, 6, 9, and 12 contain the average optimality gap. Columns 4, 7, 10, and 13 show the maximum optimality gap.

² The latest version of jORLib can be downloaded at: <http://coin-or.github.io/jorlib/>.

Table 1

Solution quality on the Coindreau et al. (2019) instances.

C	VNS			MSH			SLNS			BPC		
	BKSs	Avg. Δ	Max. Δ	BKSs	Avg. Δ	Max. Δ	BKSs	Avg. Δ	Max. Δ	BKSs	Avg. Δ	Max. Δ
20	1/10	4.09%	8.87%	10/10	0.00%	0.00%	10/10	0.00%	0.00%	10/10	0.00%	0.00%
30	0/10	3.13%	5.89%	7/10	0.01%	0.04%	8/10	0.00%	0.04%	10/10	0.00%	0.00%
40	3/10	1.23%	5.31%	6/10	0.15%	0.69%	7/10	0.22%	1.39%	10/10	0.00%	0.00%
50	2/10	2.12%	6.11%	2/10	0.91%	2.60%	4/10	0.14%	0.54%	10/10	0.00%	0.00%
Total/Avg.	6/40	2.64%	6.54%	25/40	0.27%	0.83%	29/40	0.09%	0.49%	40/40	0.00%	0.00%

Table 2

Assessing optimality on the Coindreau et al. (2019) instances.

C	VNS			MSH			SLNS			BPC		
	#Opt.	Avg. Δ	Max. Δ	#Opt.	Avg. Δ	Max. Δ	#Opt.	Avg. Δ	Max. Δ	#Opt.	Avg. Δ	Max. Δ
20	1/10	4.09%	8.87%	10/10	0.00%	0.00%	10/10	0.00%	0.00%	10/10	0.00%	0.00%
30	0/10	3.13%	5.89%	7/10	0.01%	0.04%	8/10	0.00%	0.04%	10/10	0.00%	0.00%
40	3/10	1.23%	5.31%	6/10	0.15%	0.69%	7/10	0.22%	1.39%	10/10	0.00%	0.00%
50	2/10	2.22%	6.11%	2/10	1.00%	2.60%	4/10	0.23%	0.93%	9/10	0.09%	0.91%
Total/Avg.	6/40	2.67%	6.54%	25/40	0.29%	0.83%	29/40	0.11%	0.59%	39/40	0.02%	0.23%

Table 3

Computational times on the Coindreau et al. (2019) instances.

C	VNS		MSH		SLNS		BPC		
	Avg. CPU (s)		Avg. CPU (s)		Avg. CPU (s)		Avg. CPU (s)	Min. CPU (s)	Max. CPU (s)
20	71.0		13.4		15.0		28.1	22.1	40.1
30	381.0		35.9		30.0		75.9	51.3	127.7
40	1993.0		56.1		60.0		288.7	100.2	1340.8
50	6779.0		110.5		120.0		1877.4	276.6	7200.0
Average	2306.0		54.0		56.3		567.5	112.5	2177.1

Note that BPC is the first to prove optimality for 39 (out of 40) instances. Moreover, the average optimality gap of the solutions found by BPC is 0.02%. Additionally, the maximum optimality gap on the unsolved instances is 0.91%. With regard to the metaheuristics, SLNS has the best performance finding solutions with an average optimality gap of 0.11%.

Finally, Table 3 compares the performance of each algorithm in terms of computational efficiency. Each row corresponds to an instance size. Columns 2, 3, 4, and 5 show the average runtime in seconds reported by each algorithm. Columns 6 and 7 show the minimum and maximum CPU time employed by the BPC.

While a perfect head-to-head comparison is hard to make because of differences in the programming languages and testing environment, the results suggest that in the subset of instances with 20 and 30 customers, BPC is close to match the performance of the state-of-the-art matheuristics. On average BPC uses less than 10 min.

5.3. Assessing the impact of the initialization step and the pruning strategies

As our proposed method uses a metaheuristic to initialize the set of columns, it is only logical to assess the performance of our BPC without this component. With this in mind, we ran our BPC while only using one iteration of MSH. This version of the algorithm is labeled as BPC-W. Moreover, to measure the impact of the problem-specific algorithmic components that we designed, we ran our BPC with a version of the PA that does not include the path completion and the subtour fixing strategies. This version of our BPC is labeled as BPC-O.

Table 4 compares the performance of the branch-price-and-cut algorithms described above in terms of solution quality. Each row corresponds to a combination between a version of our BPC algorithm and an instance size. Recall that each algorithm was tested on every instance across five replicates in which the initial solution is modified. Column 3 shows the average number of best-known solutions found by each algorithm. Column 4 shows the average number of optimal solutions found by each algorithm. Columns 5, 6 and 7 present the average, minimum, and maximum computational time in seconds used by each algorithm. Finally, columns 8, 9 and 10 show the average, minimum, and maximum number of columns.

Note that BPC is the only algorithm that finds the best-known solution for every instance in all the replicates. Moreover, it is the algorithm that on average provides the highest number of optimal solutions. Nevertheless, note that BPC-W is capable of finding high quality solutions and of proving optimality of at least 38 out of 40 instances. This shows that the BPC solution quality is not exclusively due to the initialization procedure. In addition, note that BPC-O is not capable of finding one of the best-known solutions before reaching the running time limit, thus showing the importance of including problem-specific pruning strategies inside the PA.

With respect to the computational times, BPC is on average faster than its counterparts, especially on the subset of instances with 40 and 50 customers, where both the initialization procedure and the additional pruning strategies improve the algorithm's

Table 4
Solution quality of the BPC variants with $\zeta = 5$ km.

Algorithm	C	Avg. #BKS	Avg. # optimal	CPU time (s)			# Columns		
				Avg.	Min	Max	Avg.	Min	Max
BPC	20	10/10	10/10	28.11	22.12	40.13	219.30	85	461
	30	10/10	10/10	75.90	51.29	127.67	393.74	92	693
	40	10/10	10/10	288.72	100.17	1340.78	1 647.88	387	6 050
	50	10/10	8.8/10	1877.42	276.60	7200.00	1 864.00	779	5 426
BPC-O	20	10/10	10/10	27.19	21.32	39.61	221.54	85	461
	30	10/10	10/10	74.07	50.60	138.73	392.72	92	664
	40	10/10	10/10	317.17	96.14	1755.69	1 562.46	486	6 824
	50	9/10	8.2/10	2026.22	279.66	7200.00	1 807.70	646	3 934
BPC-W	20	10/10	10/10	14.43	4.15	41.62	2 246.14	555	5 155
	30	10/10	10/10	118.84	28.60	342.65	12 998.62	1586	35 468
	40	10/10	10/10	1018.09	242.95	4219.65	5 871.14	1874	39 067
	50	8.8/10	8.4/10	2346.02	354.07	7200.00	10 070.38	2312	53 989

Table 5
Branch-price-and-cut performance on larger PLRP instances.

C	# Opt.	Optimality gap			CPU time (s)			Δ vs. initial solution		
		Avg.	Min	Max	Avg.	Min	Max	Avg.	Min	Max
60	3	1.62%	0.00%	3.66%	5460.14	396.02	7200.00	-2.04%	-5.83%	0.00%
70	0	3.43%	2.17%	5.39%	7200.00	7200.00	7200.00	-0.17%	-0.58%	0.00%
80	0	5.66%	4.07%	8.21%	7200.00	7200.00	7200.00	-1.21%	-5.17%	0.00%
90	0	5.93%	3.73%	8.26%	7200.00	7200.00	7200.00	-0.49%	-2.20%	0.00%
Total/Avg.	3	4.16%	2.49%	6.38%	6765.03	5499.01	7200.00	-0.98%	-3.44%	0.00%

performance. However, in the subset of instances with 20 customers, it seems that the overhead incurred by the initialization procedure does not pay off, as BPC-W is faster than BPC and delivers solutions of equivalent quality.

5.4. A new data set

In the previous experiments, we showed that our algorithm is able to optimally solve most of the 50-node instances in the PLRP set. It is worth recalling that these instances are the largest benchmarks available for our problem. To test the limits of our method, we designed a new set of larger instances. To build them, we followed the procedure described by Coindreau et al. (2019). Each instance considers a number of customers n in the set $\{60, 70, 80, 90\}$. For each value of n , we randomly generate 10 instances in which the (x,y) -coordinates of the customers and the depot are taken from a continuous uniform distribution on the intervals $[0, 10] \times [0, 10]$. In this new set of instances, the number of available workers k is set to the minimum number of workers needed to serve all the customers. This value is calculated using the left-hand side of constraint (13). All the other parameters described in Section 5.1 remain the same. The new set is publicly available at www.vrp-rep.org (VRP-REP-ID 2023-0001).

Table 5 describes the performance of the BPC algorithm on the new set of larger instances. Each row corresponds to an instance size. Column 2 shows the number of optimal solutions found by the algorithm. Columns 3, 4 and 5 report the average, minimum, and maximum optimality gap, respectively. Columns 6, 7 and 8 contain the average, minimum, and maximum CPU time employed by the BPC. Columns 9, 10 and 11 report the average, minimum, and maximum gap in the objective function of the solutions found by BPC with respect to those delivered by the initialization method, MSH. More specifically, for each instance in the set, the gap was computed as

$$\frac{f(BPC) - f(MSH)}{f(MSH)}, \quad (15)$$

where $f(\cdot)$ denotes the objective function of the solution delivered by a given approach. For these experiments, the time limit is set to two hours. All the parameters used by MSH (e.g., $T = 2500$, $Q = 60$) were fixed to their standard values (see Section 4.3).

As expected, the performance of the BPC decreases when the number of customers $|C|$ increases. Nevertheless, BPC is capable of proving optimality for 3 out of 10 instances with 60 customers within the time limit. Moreover, the average optimality gap in the subset of instances with 60 customers is 1.62%. Regarding the subset of instances with 70, 80, and 90 customers, BPC is not capable of closing the optimality gap for any of the instances within the time limit. While performing our experiments, we noticed that most of the time spent by BPC is used to try to improve the lower bound. Indeed, in this new set of instances, the average improvement of the solutions found by MSH is only 0.98%. Thus one possible direction to extend the reach of our method to 70-plus-customer instances would be to devise new valid inequalities to strengthen the lower bound faster. For completeness, Table B.11 in Appendix B shows the objective function value of the solutions found by MSH and BPC on each individual instance. It also reports the lower bound found by BPC before reaching the time limit.

Table 6
Performance of the CG algorithms solving the RSCP root node with $\zeta = 0$.

CG algorithm	C	Lower bound			CPU time (s)			# Columns		
		Avg.	Min	Max	Avg.	Min	Max	Avg.	Min	Max
CG-PA	20	41.85	33.64	46.01	21.53	17.61	24.66	321.00	117	827
	30	52.35	47.91	55.84	43.49	37.20	49.84	538.06	167	1046
CG-NG-10	20	41.84	33.64	46.01	11.38	10.37	13.30	548.30	125	2592
	30	52.22	47.78	55.84	21.72	18.58	26.35	769.04	215	1762
CG-NG-7	20	41.75	33.43	46.01	11.53	10.34	15.38	758.88	171	3565
	30	52.11	47.49	55.54	21.90	18.88	26.08	938.98	209	2364

Table 7
Performance of the pricing algorithms solving the root node with $\zeta = 5$ km.

CG algorithm	C	Lower bound			CPU time (s)			# Columns		
		Avg.	Min	Max	Avg.	Min	Max	Avg.	Min	Max
CG-PA	20	36.64	29.61	42.28	25.79	21.10	30.95	194.06	90	503
	30	45.16	40.64	49.59	61.02	50.78	78.54	349.54	84	672
CG-NG-10	20	36.62	29.61	42.28	35.94	21.36	69.54	287.70	95	666
	30	45.08	40.21	49.59	370.70	127.06	918.51	475.64	72	1036
CG-NG-7	20	36.51	28.90	42.28	32.81	19.23	63.07	349.28	89	641
	30	44.89	40.00	49.59	273.68	94.67	749.04	591.86	78	1195

5.5. Analyzing the pricing problem algorithm

Most modern BPC algorithms for vehicle routing solve the pricing problem using labeling algorithms that rely on the NG-path relaxation proposed by Baldacci et al. (2011). This relaxation consists in defining a neighborhood $\mathcal{N}_i$ for every customer $i \in C$ that includes the X closest customers to i and the customer itself. An NG-path can include cycles starting and ending at customer j if and only if there exists a customer i in the cycle such that $j \in \mathcal{N}_i$. Thus, such a cycle is forbidden if and only if $j \in \mathcal{N}_i$ for every customer i it contains. The higher the value of X , the tighter the relaxation will be. Due to its success in column generation based algorithms, we believe it is interesting to compare the performance of our adaptation of the PA against a classical single-directional labeling algorithm implementing the NG-path relaxation. The implemented labeling algorithm uses dominance rules to discard labels. It also exploits an adapted version of the infeasibility, bounds, and rollback pruning strategies. Moreover, during the first iterations, the labeling algorithm is applied heuristically following the same logic described in Section 4.2. In addition, at every iteration, we first relax the dominance rules by ignoring conditions (A.4)–(A.5). An in-depth description of the algorithm is available in Appendix A.

To compare the performance of the PA and the labeling algorithm, we solve the RSCP without applying any cuts or branching decisions. In the remainder of this section, the version of the algorithm using the PA to solve the pricing problem is labeled CG-PA, and that using the labeling algorithm is labeled CG-NG- X . For these experiments, we solve the instances with 20 and 30 customers under two settings. First, the case in which walking is not allowed, namely $\zeta = 0$ km. Second, the case in which walking is allowed, namely $\zeta = 5$ km. We consider these two settings to isolate the impact of including park-and-loop routes on both pricing algorithms.

Table 6 compares the performance of each column generation algorithm while solving the RSCP when walking is not allowed. Each row in the table corresponds to a pricing algorithm and a number of customers. Columns 3, 4 and 5 show the average, minimum, and maximum lower bound produced by each approach. Columns 6, 7 and 8 present the average, minimum, and maximum computational time in seconds. Finally, columns 9, 10 and 11 show the average, minimum, and maximum number of columns found by the corresponding pricing algorithm.

As the results show, CG-PA provides on average a better (higher) lower bound than the CG algorithms that use the NG-path relaxation. However, in this subset of instances the CG algorithms using the NG-path relaxation solve the RSCP faster. With respect to the number of columns, CG-PA requires on average a lower number of columns. These results are expected, as both CG-NG-10 and CG-NG-7 are solving a relaxation of the pricing problem. Moreover, without the presence of walking subtours, the subtour fixing strategy used by the PA does not have an impact on the algorithm's performance.

Table 7 compares the performance of each column generation algorithm while solving the RSCP when walking is allowed. Similarly, each row in the table corresponds to a pricing algorithm and a number of customers. Columns 3, 4 and 5 show the average, minimum, and maximum lower bound delivered by each approach. Columns 6, 7 and 8 present the average, minimum, and maximum computational time in seconds. Finally, columns 9, 10 and 11 show the average, minimum, and maximum number of columns found by the corresponding pricing algorithm.

Note that the CG algorithms using the NG-paths relaxation provide lower bounds of lower quality in comparison with the bounds found by CG-PA. This can have a large impact on the solution algorithm as branch-and-bound nodes can be pruned early. Moreover, the computational time required to solve the RSCP is significantly larger for the NG-paths based algorithms. Indeed, considering the subset of instances with 30 customers, on average CG-PA takes 61.02 s to solve the RSCP while CG-NG-10 and CG-NG-7 take 370.70 and 270.68 s, respectively. Although we can only speculate, we suspect there are two main reasons for these differences

Table 8
Performance of BPC algorithms on the Coindreau et al. (2019) instances.

C	BPC		BPC-NG-7				
	# Opt.	# BKS	# Opt.	# BKS	Avg. CPU (s)	Min. CPU (s)	Max. CPU (s)
20	10/10	10/10	10/10	10/10	182.52	25.92	1218.00
30	10/10	10/10	7/10	9/10	2795.20	129.32	7200.00
40	10/10	10/10	2/10	6/10	6572.29	2640.12	7200.00
50	9/10	10/10	0/10	0/10	7200.00	7200.00	7200.00
Total/Avg.	39/40	40/40	19/40	25/40	4187.50	2498.84	5704.50

Table 9
Average driven distance while varying the maximum walking distance.

# Customers	VRP	PLRP ($\zeta = 5$)		PLRP ($\zeta = 10$)	
	Avg. km	Avg. km	% Δ vs. VRP	Avg. km	% Δ vs. VRP
20	42.08	36.71	−12.8%	32.63	−22.5%
30	53.35	45.32	−15.0%	38.04	−28.7%
40	60.82	56.84	−6.5%	54.97	−9.6%
50	69.38	61.80	−10.9%	60.43	−12.9%
Average	56.41	50.17	−11.3%	46.52	−18.4%

in performance. First, recall that the PA executes a depth-first search, while the labeling algorithm performs a breadth-first search. Thus, the PA tends to find promising routes faster and, as such, it significantly benefits from the increased strength of the bounds pruning strategy. Secondly, as opposed to the PA, the labeling algorithm relies heavily on dominance rules in order to discard labels. However, when walking subtours are allowed, labels are only comparable if they have the same parking status (i.e., same parking spot), which is often not the case, especially when the number of customers is high. As a result, in our experiments we observed that the size of the labels queue considerably grows in comparison with the case in which walking is not allowed.

To push the analysis forward, we implemented a BPC algorithm that uses the labeling algorithm to solve the pricing problem and reuses all the branching and cutting machinery in our BPC algorithm. For quick reference, we denote this alternative method as BPC-NG-X. We ran experiments on the Coindreau et al. (2019) instances setting X equal to 7. All other parameters for both two algorithms (e.g., $\varphi = 5$, $\varepsilon = 0.1$, $Y = 10$) were fixed to their standard values (see Section 5.1). Table 8 describes the performance of both BPC algorithms. Each row corresponds to an instance size. Columns 2 and 4 show the number of instances in which the corresponding BPC algorithm was able to prove optimality. Columns 3 and 5 show the number of best-known solutions found by each BPC algorithm. Columns 6, 7 and 8 contain the average, minimum, and maximum computational time used by BPC-NG-7. The computational times of BPC are shown in Table 4.

As the results show, our BPC outperforms BPC-NG-7 in both solution quality and running times. More specifically, BPC-NG-7 only proves optimality for 19/40 instances while BPC proves optimality for 39/40 instances. Moreover, BPC-NG-7 is only capable of matching 25/40 of the best-known solutions within the time limit. Regarding the computational times, BPC-NG-7 uses on average 4187.50 s to solve PLRP instances while BPC only uses 567.5. Thus, the performance drop observed by changing the algorithm that solves the pricing problem is significant. Furthermore, it seems that BPC-NG-7 particularly struggles on the subset of instances with 50 customers, in which even matching the best-known solution can be challenging.

5.6. Assessing the importance of introducing park-and-loop routes

Introducing park-and-loop routes helps to decrease the driven distance. To assess their potential impact, we compare the objective function of the best-known solutions while varying the maximum walking distance. Namely, we consider the vehicle routing configuration (i.e., $\zeta = 0$) and the park-and-loop configuration with ζ equal to 5 or 10 km.

Table 9 compares the average driven distance of each configuration. Each row corresponds to an instance size. Column 2 shows the average driven distance without allowing walking subtours. Columns 3 and 5 show the average driven distance while imposing a limit of 5 or 10 km on the total walking distance respectively. Finally, Columns 4 and 6 show the average savings gained by introducing walking subtours.

As the results show, introducing walking subtours decreases the driven distance by 11.3% and 18.4% on average. As expected, increasing the walking distance limit allows for larger walking subtours and, in turn, for greater savings. Fig. 3 shows the best-known solutions for instance 30_A_1. When walking subtours are not allowed, the total driven distance is 48.81 km. If walking subtours are allowed, the driven distances are 40.73 km and 32.93 km while setting ζ to 5 and 10 km, respectively. As stated before, major savings can be achieved by introducing park-and-loop routes. As this figure shows, increasing the maximum walking distance has an impact on the size and length of the walking subtours present in each route.

We have developed a website available at <https://chairelogistique.hec.ca/en/scientific-data/> where researchers can download the instances, access instance-by-instance results, and upload their own solutions in order to encourage future research on this problem and make comparisons with our results easier.

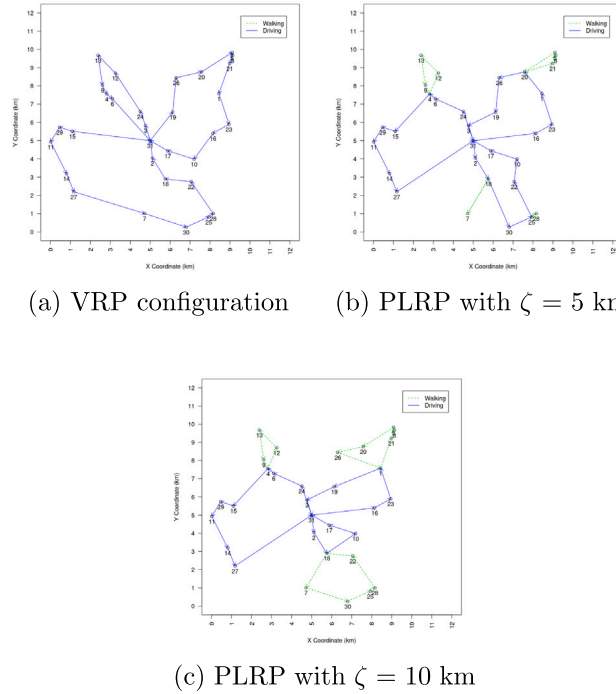


Fig. 3. Solutions for instance 30_A.1.

6. Concluding remarks

In this paper, we presented a branch-price-and-cut algorithm for solving the PLRP. To do so, we formulated the PLRP as a set covering problem that considers a large set of paths. To solve this model, we use a column generation approach that unveils promising paths that favor the objective of minimizing the total cost. The master problem selects paths to serve all customers, while the pricing problem generates feasible paths. We leveraged a tailored and improved version of the pulse algorithm to solve an elementary resource constrained shortest path with park-and-loop, that includes additional pruning strategies and allows for considering the subset row inequalities. Using this new version of the pulse algorithm improves the performance of the branch-price-and-cut algorithm.

We compared our branch-price-and-cut algorithm with the state-of-the-art algorithms for solving a related problem called the vehicle routing problem with transportable resources without carpooling. The proposed algorithm was capable of finding all the previously best-known solutions. Moreover, it found 11 previously unknown optimal solutions. In addition, our method is the first capable of solving 39 instances to optimality out of the 40 instances that composed the testbed. Our experiments also show the advantages of providing a high quality pool of columns as a warm start as it significantly decreases the computational effort required by the algorithm.

We also showed the benefits of introducing a park-and-loop structure in the routing plan, as it allows decreasing the total driven distance. To encourage future research on the PLRP, we developed an online tool that allows the members of the community to visualize the best-known solutions and upload their own solutions for checking and plotting. Future research should focus on extending the branch-price-and-cut algorithm to solve the PLRP with time windows. Moreover, improving the performance of the pricing problem by implementing bidirectional search strategies and other problem-specific pruning strategies seems promising to decrease the computational time needed by each CG iteration.

CRedit authorship contribution statement

Nicolás Cabrera: Conceptualization, Methodology, Software, Formal analysis, Investigation, Writing – original draft. **Jean-François Cordeau:** Conceptualization, Methodology, Resources, Writing – review & editing, Supervision, Funding acquisition, Project administration. **Jorge E. Mendoza:** Conceptualization, Methodology, Resources, Writing – review & editing, Supervision, Funding acquisition, Project administration.

Acknowledgments

This research was partially funded by the Canada First Research Excellence Fund through IVADO and the Discovery grants. This research was also co-funded by HEC Montréal through the Chair in Logistics and Transportation and the Research Professorship on Clean Transportation Analytics. The authors are also grateful to two anonymous referees for their valuable comments which helped improve the quality of the paper.

Appendix A. Labeling algorithm: In-depth description

In this section, we present a labeling algorithm to solve the pricing problem described in Section 3.2. Our algorithm is partially based on the ideas presented by Feillet et al. (2004) and Baldacci et al. (2011).

The algorithm extends labels ℓ (i.e., partial paths) starting at $\bar{0}$ and ending at a given node $n(\ell)$. A label ℓ stores the cumulative reduced cost $r(\ell)$, the cumulative time $t(\ell)$, the cumulative walking distance $w(\ell)$, the parking spot $p(\ell)$, a resource $\Pi_i^u \in \{0, 1\}$ that takes the value of one if customer $i \in C$ cannot be visited by the partial path (due to time limit and walking distance constraints), and a resource $\Pi_i^{ng} \in \{0, 1\}$ that takes the value of one if a customer $i \in C$ is forbidden by the ng-paths cycling restrictions described in Section 5.5. If the vehicle is not currently parked, $p(\ell) = -1$. Whenever a label has a resource consumption that exceeds the corresponding resource availability it is immediately discarded. Labels are also discarded if they do not comply with the ng-paths cycling restrictions or if they are dominated by other labels. We also discard labels using the rollback and the bounds pruning strategies described in Section 3.3. All the labels successfully extended to node $\bar{0}$ can be added to the MP as they represent feasible routes with negative reduced costs.

The main logic of the algorithm is presented in Algorithm 6. Line 1 runs the bounding procedure given the bound step size and the bounding time limits. To the best of our knowledge, this is the first time that the pulse algorithm is included as a component of a labeling algorithm. Line 2 initializes the labels queue Q with a label ℓ representing the source depot (i.e., $n(\ell) = \bar{0}$) such that $r(\ell) = 0$, $t(\ell) = 0$, $w(\ell) = 0$, $p(\ell) = -1$, and $\Pi_i^u \in \{0, 1\} = \Pi_i^{ng} \in \{0, 1\} = 0$ for all customers $i \in C$. Line 3 initializes the set of negative reduced cost routes $\mathcal{H}$. From lines 4 to 9, the algorithm retrieves and expands non-dominated labels until Y routes with negative reduced cost have been found or until the queue of labels is empty. Line 5 retrieves the first label from the queue. Line 6 checks if the label is not dominated by another label at the same node. Line 7 checks if the label can be discarded using a bound on the best reduced cost that can be achieved given the current time consumption. Line 8 checks if the label can be discarded using the rollback pruning strategy. If the label is not discarded, line 9 extends the label through all the outgoing arcs of the current node and adds the new labels to Q . Line 14 returns all the routes at the end depot with negative reduced cost stored in $\mathcal{H}$.

Algorithm 6 Labeling algorithm

Require: $\mathcal{G}'$, directed multi graph; ϕ , duration limit; ζ , walking distance limit; $\bar{0}$, start node; $\bar{0}$, end node; Δ , bound step size; $[\bar{t}, \underline{t}]$, bounding time limits; ℓ , initial label.

```

1: bound( $\mathcal{G}'$ ,  $\Delta$ ,  $[\bar{t}, \underline{t}]$ ) ▷ see Section 3.3.2
2: push( $Q$ ,  $\ell$ )
3:  $\mathcal{H} \leftarrow \emptyset$ 
4: while  $Q \neq \emptyset \wedge |\mathcal{H}| \leq Y$  do
5:    $\ell \leftarrow \text{pop}(Q)$ 
6:   if  $\neg \text{dominance}(\ell)$  then ▷ see Appendix A.1
7:     if  $\neg \text{bounds}(n(\ell), r(\ell), t(\ell))$  then ▷ see Section 3.3.2
8:       if  $\neg \text{rollback}(n(\ell), r(\ell), t(\ell), w(\ell))$  then ▷ see Section 3.3.3
9:          $Q \leftarrow Q \cup \text{extend}(\ell)$  ▷ see Appendix A.2
10:      end if
11:    end if
12:  end if
13: end while
14: return  $\mathcal{H}$ 

```

A.1. Dominance rules

During the execution of the algorithm, several labels representing different partial paths will be expanded to a given node $i \in \mathcal{V}'$. With this in mind, we define dominance relations between two labels ℓ and ℓ' at a current node $n(\ell) = n(\ell')$. More precisely, label ℓ dominates label ℓ' if the following conditions hold:

$$p(\ell) = p(\ell'), \quad (\text{A.1})$$

$$r(\ell) \leq r(\ell'), \quad (\text{A.2})$$

$$t(\ell) \leq t(\ell'), \quad (\text{A.3})$$

$$w(\ell) \leq w(\ell'), \quad (\text{A.4})$$

$$\max \{ \Pi_i^{ng}(\ell), \Pi_i^u(\ell) \} \leq \max \{ \Pi_i^{ng}(\ell'), \Pi_i^u(\ell') \} \quad \forall i \in C. \quad (\text{A.5})$$

Conditions (A.1) ensure that the two labels represent partial paths with equal parking status. Conditions (A.2)–(A.5) imply that any feasible extension of the dominated label ℓ' is also a feasible extension of label ℓ with an equal or better reduced cost. During the first iterations of the BPC procedure, it is possible to ignore conditions (A.4)–(A.5) to accelerate the solution of the pricing problem.

The previous dominance rules are only valid if the master problem does not include a subset row inequality (SRI). To account for the presence of SRIs, we modified the dominance rules following the ideas presented by Jepsen et al. (2008). More specifically, let $v_S(\ell)$ denote the number of times modulo 2 that label ℓ has served the customers in $S \subseteq C$. When extending label ℓ through arc $(i, j) \in \mathcal{A}'$ to create label ℓ' , we set $v_S(\ell') = v_S(\ell)$ if $j \notin S$ and $v_S(\ell') = (v_S(\ell) + 1) \bmod 2$ otherwise. If, $v_S(\ell') = 0$ and $v_S(\ell) = 1$ we subtract the dual variable β_S from $r(\ell')$. Then, the dominance rules are modified by replacing condition (A.2) with:

$$r(\ell) - \sum_{S \subseteq C | v_S(\ell) > v_S(\ell')} \beta_S \leq r(\ell'), \quad (\text{A.6})$$

Condition (A.6) ensures that $r(\ell)$ is sufficiently less than $r(\ell')$ to compensate for any possible additional penalties.

A.2. Label extension

A label is only extended if $t(\ell) \leq \phi$ and $w(\ell) \leq \zeta$. When a label ℓ is extended through arc $(i, j) \in \mathcal{A}'$ to create label ℓ' the resources are updated depending on the operation performed. There are four types of label extensions: (1) driving, (2) parking the vehicle and walking, (3) walking without unpark, and (4) walking with unpark. In what follows, we will describe the extension rules in detail for each case.

Case 1: Driving. In this case, the route is extended by driving between nodes i and j . This extension is only possible if $p(\ell) = -1$ and $(i, j) \in \mathcal{A}^1 \cup \mathcal{A}^3$ (i.e., through a driving arc). Label ℓ' has the following attributes:

$$p(\ell') = p(\ell), \quad (\text{A.7})$$

$$r(\ell') = r(\ell) + r_{ij}, \quad (\text{A.8})$$

$$t(\ell') = t(\ell) + \eta_{ij} + s_j, \quad (\text{A.9})$$

$$w(\ell') = w(\ell), \quad (\text{A.10})$$

$$\Pi_m^u(\ell') = \begin{cases} 0, & t(\ell') + \eta_{jm} + s_m \leq \phi \mid (j, m) \in \mathcal{A}^1; \\ 1, & \text{otherwise;} \end{cases} \quad \forall m \in C, \quad (\text{A.11})$$

$$\Pi_m^{ng}(\ell') = \begin{cases} 1, & (m = j) \vee (\Pi_m^{ng}(\ell) = 1 \wedge m \in \mathcal{N}_j); \\ 0, & \text{otherwise;} \end{cases} \quad \forall m \in C. \quad (\text{A.12})$$

Case 2: Parking the vehicle and walking. In this case, the route is extended by walking between nodes i and j after parking the vehicle at node i . This extension is only possible if $p(\ell) = -1$ and $(i, j) \in \mathcal{A}^2 \cup \mathcal{A}^4$ (i.e., through a walking arc). Label ℓ' has the following attributes:

$$p(\ell') = i, \quad (\text{A.13})$$

$$r(\ell') = r(\ell) + r_{ij}, \quad (\text{A.14})$$

$$t(\ell') = t(\ell) + \eta_{ij} + s_j, \quad (\text{A.15})$$

$$w(\ell') = w(\ell) + \delta_{ij}, \quad (\text{A.16})$$

$$\Pi_m^u(\ell') = \begin{cases} 0, & t(\ell') + \eta_{jm} + s_m + \eta_{mi} \leq \phi \mid (j, m), (m, i) \in \mathcal{A}^2; \\ 1, & \text{otherwise;} \end{cases} \quad \forall m \in C, \quad (\text{A.17})$$

$$\Pi_m^{ng}(\ell') = \begin{cases} 1, & (m = j) \vee (\Pi_m^{ng}(\ell) = 1 \wedge m \in \mathcal{N}_j); \\ 0, & \text{otherwise;} \end{cases} \quad \forall m \in C. \quad (\text{A.18})$$

Case 3: Walking without unpark. In this case, the route is extended by walking between nodes i and j . The vehicle remains parked at node $p(\ell)$. This extension is only possible if $p(\ell) \neq -1$ and $(i, j) \in \mathcal{A}^2 \cup \mathcal{A}^4$ (i.e., through a walking arc). Label ℓ' has the following attributes:

$$p(\ell') = p(\ell), \quad (\text{A.19})$$

Table B.10
Detailed results on the [Coindreau et al. \(2019\)](#) instances.

Instance	VNS	MSH	SLNS	BPC	BKS
20_A_1	32.4037	30.9482	30.9482	30.9482	30.9482
20_A_2	41.6349	41.5547	41.5547	41.5547	41.5547
20_A_3	39.4469	36.2344	36.2344	36.2344	36.2344
20_A_4	39.1459	36.0251	36.0251	36.0251	36.0251
20_A_5	35.2747	35.2747	35.2747	35.2747	35.2747
20_A_6	43.7627	42.2866	42.2866	42.2866	42.2866
20_A_7	40.1143	38.6881	38.6881	38.6881	38.6881
20_A_8	39.1395	36.8504	36.8504	36.8504	36.8504
20_A_9	29.9427	29.6105	29.6105	29.6105	29.6105
20_A_10	41.2404	39.6656	39.6656	39.6656	39.6656
30_A_1	41.2853	40.7372	40.7372	40.7372	40.7372
30_A_2	46.6485	45.6635	45.6465	45.6465	45.6465
30_A_3	50.9785	49.1828	49.1828	49.1828	49.1828
30_A_4	46.3310	43.7556	43.7556	43.7556	43.7556
30_A_5	48.3961	47.0619	47.0619	47.0619	47.0619
30_A_6	51.6505	49.5880	49.5880	49.5880	49.5880
30_A_7	45.5537	45.5537	45.5537	45.5340	45.5340
30_A_8	45.3691	43.6799	43.6799	43.6799	43.6799
30_A_9	41.8553	41.1629	41.1629	41.1609	41.1609
30_A_10	49.5241	46.8936	46.8936	46.8936	46.8936
40_A_1	59.1695	59.1695	59.1695	59.1695	59.1695
40_A_2	58.5186	58.5186	58.5186	58.1191	58.1191
40_A_3	63.1723	62.9222	62.9222	62.9222	62.9222
40_A_4	50.3730	50.3730	50.3730	50.3730	50.3730
40_A_5	52.7504	51.7278	51.7278	51.7278	51.7278
40_A_6	63.0658	61.2621	61.2621	61.2621	61.2621
40_A_7	57.7673	54.8824	55.6158	54.8554	54.8554
40_A_8	56.3737	56.3140	55.9774	55.9774	55.9774
40_A_9	56.4612	56.3871	56.3871	56.3013	56.3013
40_A_10	57.6954	57.6954	57.6954	57.6954	57.6954
50_A_1	60.5063	57.0235	57.0235	57.0235	57.0235
50_A_2	62.4825	62.1686	60.6031	60.5906	60.5906
50_A_3	65.8727	63.8855	63.6788	63.5529	63.5529
50_A_4	58.2333	57.9472	56.9391	56.6342	56.6342
50_A_5	64.1751	64.0909	64.0909	64.0909	64.0909
50_A_6	66.1254	65.0105	65.0105	64.8116	64.8116
50_A_7	63.6674	63.6674	63.6674	63.6350	63.6350
50_A_8	70.9057	69.7911	68.8367	68.6323	68.6323
50_A_9	58.2212	58.7191	58.2212	58.2212	58.2212
50_A_10	60.7796	61.2239	60.7796	60.7796	60.7796

$$r(\ell') = r(\ell) + r_{ij}, \quad (\text{A.20})$$

$$t(\ell') = t(\ell) + \eta_{ij} + s_j, \quad (\text{A.21})$$

$$w(\ell') = w(\ell) + \delta_{ij}, \quad (\text{A.22})$$

$$\Pi_m^u(\ell') = \begin{cases} 0, & t(\ell') + \eta_{jm} + s_m + \eta_{mp(\ell)} \leq \phi \mid (j, m), (m, p(\ell)) \in \mathcal{A}^2; \\ 1, & \text{otherwise;} \end{cases} \quad \forall m \in C, \quad (\text{A.23})$$

$$\Pi_m^{ng}(\ell') = \begin{cases} 1, & (m = j) \vee (\Pi_m^{ng}(\ell) = 1 \wedge m \in \mathcal{N}_j); \\ 0, & \text{otherwise;} \end{cases} \quad \forall m \in C. \quad (\text{A.24})$$

Case 4: *Walking with unpark*. In this case, the route is extended by walking between nodes i and j . The vehicle is recovered at node $p(\ell)$. This extension is only possible if the $p(\ell) \neq -1$ and $(i, j) \in \mathcal{A}^2 \cup \mathcal{A}^4$ (i.e., through a walking arc). Label ℓ_j has the following attributes:

$$p(\ell') = -1, \quad (\text{A.25})$$

$$r(\ell') = r(\ell) + r_{ij} + \pi_j, \quad (\text{A.26})$$

$$t(\ell') = t(\ell) + \eta_{ij}, \quad (\text{A.27})$$

Table B.11
Detailed results on the PLRP set of large instances.

Instance	k	MSH	BPC	LB
60_A_1	5	63.6180	63.6180	61.6495
60_A_2	5	67.9332	64.8337	63.1645
60_A_3	5	66.7018	64.3194	63.3934
60_A_4	5	69.4019	68.8561	68.8561
60_A_5	5	72.0505	67.8520	67.4220
60_A_6	5	65.7642	65.6751	64.3115
60_A_7	5	65.4693	64.7246	64.7246
60_A_8	5	66.4684	65.6574	65.6574
60_A_9	5	69.3708	67.1968	65.3779
60_A_10	6	77.4000	77.3963	74.5644
70_A_1	6	80.7800	80.3145	77.4835
70_A_2	5	85.1100	85.1052	81.5012
70_A_3	6	71.4200	71.4182	69.7106
70_A_4	6	74.8472	74.4721	71.8464
70_A_5	5	79.0669	79.0669	76.9217
70_A_6	6	70.8563	70.8563	69.3213
70_A_7	6	78.2697	78.2697	76.0983
70_A_8	6	75.8930	75.8930	71.8024
70_A_9	5	80.0280	79.8592	75.8490
70_A_10	6	82.5269	82.1809	80.0585
80_A_1	6	92.0970	91.4923	86.1695
80_A_2	6	92.2233	90.9719	86.9094
80_A_3	6	96.9931	96.2431	90.0295
80_A_4	6	99.6504	94.4960	89.9932
80_A_5	6	93.4427	93.4428	86.3854
80_A_6	6	94.6270	91.1502	87.4419
80_A_7	6	90.1759	90.1759	86.4039
80_A_8	6	92.4352	92.4352	86.7532
80_A_9	6	101.2854	101.0565	92.7555
80_A_10	6	93.1675	92.9522	88.3396
90_A_1	7	92.1341	92.1341	88.6969
90_A_2	7	85.6971	85.6971	80.8916
90_A_3	7	88.6107	88.6107	81.9134
90_A_4	7	94.0196	91.9519	87.9134
90_A_5	7	100.7159	100.3967	94.3991
90_A_6	7	81.6312	81.5321	76.5057
90_A_7	7	95.2463	95.2463	88.8707
90_A_8	7	97.8633	96.7539	91.6895
90_A_9	7	85.2391	85.2391	78.1943
90_A_10	7	99.6994	98.5392	92.9275

$$w(\ell') = w(\ell) + \delta_{ij}, \quad (\text{A.28})$$

$$\Pi_m^u(\ell') = \begin{cases} 0, & t(\ell') + \eta_{jm} + s_m \leq \phi \mid (j, m) \in \mathcal{A}^1; \\ 1, & \text{otherwise;} \end{cases} \quad \forall m \in C, \quad (\text{A.29})$$

$$\Pi_m^{ng}(\ell') = \begin{cases} 1, & (m = j) \vee (\Pi_m^{ng}(\ell) = 1 \wedge m \in \mathcal{N}_j); \\ 0, & \text{otherwise;} \end{cases} \quad \forall m \in C. \quad (\text{A.30})$$

Appendix B. Detailed results for each instance

Table B.10 shows the performance of each algorithm on each instance of the VRPTR without carpooling. Each row corresponds to an instance. Columns 2 to 5 show the objective function of the solution found by the VNS, MSH, SLNS, and the BPC algorithm respectively. Column 6 shows the objective function of the current best-known solution.

Table B.11 reports the solutions found by MSH and BPC to the set of large instances. Each row corresponds to an instance. Column 2 reports the number of available workers k . Columns 3 and 4 show the objective function of the solution found by MSH and BPC. Column 5 reports the lower bound found by BPC.

References

- Arslan, O., Jabali, O., Laporte, G., 2018. Exact solution of the evasive flow capturing problem. *Oper. Res.* 66 (6), 1625–1640.
 Baldacci, R., Mingozzi, A., Roberti, R., 2011. New route relaxation and pricing strategies for the vehicle routing problem. *Oper. Res.* 59 (5), 1269–1283.

- Belenguer, J., Benavent, E., Martínez, A., Prins, C., Prodhon, C., Villegas, J., 2016. A branch-and-cut algorithm for the single truck and trailer routing problem with satellite depots. *Transp. Sci.* 50, 735–749.
- Bolívar, M.A., Lozano, L., Medaglia, A.L., 2014. Acceleration strategies for the weight constrained shortest path problem with replenishment. *Optim. Lett.* 8 (8), 2155–2172.
- Cabrera, N., Cordeau, J.-F., Mendoza, J.E., 2022. The doubly open park-and-loop routing problem. *Comput. Oper. Res.* 143, 105761.
- Cabrera, N., Medaglia, A.L., Lozano, L., Duque, D., 2020. An exact bidirectional pulse algorithm for the constrained shortest path. *Networks* 76 (2), 128–146.
- Chao, I.-M., 2002. A tabu search method for the truck and trailer routing problem. *Comput. Oper. Res.* 29, 33–51.
- Coindreau, M., Galloway, O., Zufferey, N., 2019. Vehicle routing with transportable resources: Using carpooling and walking for on-site services. *European J. Oper. Res.* 279, 996–1010.
- Contardo, C., Martinelli, R., 2014. A new exact algorithm for the multi-depot vehicle routing problem under capacity and route length constraints. *Discrete Optim.* 12, 129–146.
- Corredor-Montenegro, D., Cabrera, N., Akhavan-Tabatabaei, R., Medaglia, A.L., 2021. On the shortest α -reliable path problem. *Top* 29 (1), 287–318.
- Costa, L., Contardo, C., Desaulniers, G., 2019. Exact branch-price-and-cut algorithms for vehicle routing. *Transp. Sci.* 53 (4), 946–985.
- Derigs, U., Pullmann, M., Vogel, U., 2013. Truck and trailer routing - problems, heuristics and computational experience. *Comput. Oper. Res.* 40, 536–546.
- Desaulniers, G., Desrosiers, J., Solomon, M.M., 2005. *Column Generation*. Springer Science & Business Media.
- Duque, D., Lozano, L., Medaglia, A.L., 2014. Solving the orienteering problem with time windows via the pulse framework. *Comput. Oper. Res.* 54, 168–176.
- Duque, D., Lozano, L., Medaglia, A.L., 2015. An exact method for the biobjective shortest path problem for large-scale road networks. *European J. Oper. Res.* 242 (3), 788–797.
- Feillet, D., Dejax, P., Gendreau, M., Gueguen, C., 2004. An exact algorithm for the elementary shortest path problem with resource constraints: Application to some vehicle routing problems. *Networks* 44 (3), 216–229.
- Jepsen, M., Petersen, B., Spoorendonk, S., Pisinger, D., 2008. Subset-row inequalities applied to the vehicle-routing problem with time windows. *Oper. Res.* 56, 497–511.
- Le Colleter, T., Dumez, D., Lehuédé, F., Péton, O., 2023. Small and large neighborhood search for the park-and-loop routing problem with parking selection. *European J. Oper. Res.* 308 (3), 1233–1248.
- Li, H., Wang, H., Chen, J., Bai, M., 2020. Two-echelon vehicle routing problem with time windows and mobile satellites. *Transp. Res. B* 138, 179–201.
- Lin, S., Yu, V., Chou, S., 2009. Solving the truck and trailer routing problem based on a simulated annealing heuristic. *Comput. Oper. Res.* 36, 1683–1692.
- Lozano, L., Duque, D., Medaglia, A.L., 2016. An exact algorithm for the elementary shortest path problem with resource constraints. *Transp. Sci.* 50 (1), 348–357.
- Lozano, L., Smith, J.C., 2017. A backward sampling framework for interdiction problems with fortification. *INFORMS J. Comput.* 29 (1), 123–139.
- Lübbecke, M.E., Desrosiers, J., 2005. Selected topics in column generation. *Oper. Res.* 53 (6), 1007–1023.
- Marques, G., Sadykov, R., Deschamps, J.-C., Dupas, R., 2020. An improved branch-cut-and-price algorithm for the two-echelon capacitated vehicle routing problem. *Comput. Oper. Res.* 114, 104833.
- Martínez-Sykora, A., McLeod, F., Lamas-Fernandez, C., Bektaş, T., Cherrett, T., Allen, J., 2020. Optimised solutions to the last-mile delivery problem in London using a combination of walking and driving. *Ann. Oper. Res.* 295 (2), 645–693.
- Montoya, A., Guéret, C., Mendoza, J.E., Villegas, J.G., 2016. A multi-space sampling heuristic for the green vehicle routing problem. *Transp. Res. C* 70, 113–128.
- Parragh, S.N., Cordeau, J.-F., 2017. Branch-and-price and adaptive large neighborhood search for the truck and trailer routing problem with time windows. *Comput. Oper. Res.* 83, 28–44.
- Pessoa, A., Sadykov, R., Uchoa, E., Vanderbeck, F., 2013. In-out separation and column generation stabilization by dual price smoothing. In: Bonifaci, V., Demetrescu, C., Marchetti-Spaccamela, A. (Eds.), *Experimental Algorithms*. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 354–365.
- Reed, S., Campbell, A.M., Thomas, B.W., 2022. Does parking matter? The impact of search time for parking on last-mile delivery optimization. *arXiv:2107.06788*.
- Restrepo, M.I., Lozano, L., Medaglia, A.L., 2012. Constrained network-based column generation for the multi-activity shift scheduling problem. *Int. J. Prod. Econ.* 140 (1), 466–472.
- Rothembächer, A.-K., Drexl, M., Irnich, S., 2018. Branch-and-price-and-cut for the truck-and-trailer routing problem with time windows. *Transp. Sci.* 52 (5), 1174–1190.
- Schrotenboer, A.H., Ursavas, E., Vis, I.F., 2019. A branch-and-price-and-cut algorithm for resource-constrained pickup and delivery problems. *Transp. Sci.* 53 (4), 1001–1022.
- Semet, F., 1995. A two-phase algorithm for the partial accessibility constrained vehicle routing problem. *Ann. Oper. Res.* 61, 45–65.
- Sheuerer, S., 2006. A tabu search heuristic for the truck and trailer routing problem. *Comput. Oper. Res.* 33, 894–909.
- Tamke, F., Buscher, U., 2021. A branch-and-cut algorithm for the vehicle routing problem with drones. *Transp. Res. B* 144, 174–203.
- Villegas, J., Prins, C., Medaglia, A., Velasco, N., 2010. GRASP/VND and multi-start evolutionary local search for the single truck and trailer routing problem with satellite depots. *Eng. Appl. Artif. Intell.* 23, 780–794.
- Villegas, J., Prins, C., Medaglia, A., Velasco, N., 2013. A matheuristic for the truck and trailer routing problem. *European J. Oper. Res.* 230, 231–244.
- Villegas, J., Prins, C., Prodhon, C., Medaglia, A., Velasco, N., 2011. A GRASP with evolutionary path relinking for the truck and trailer routing problem. *Comput. Oper. Res.* 38, 1319–1334.
- Yamín, D., Medaglia, A.L., Prakash, A.A., 2022. Exact bidirectional algorithm for the least expected travel-time path problem on stochastic and time-dependent networks. *Comput. Oper. Res.* 141, 105671.
- Yin, Y., Li, D., Wang, D., Ignatius, J., Cheng, T., Wang, S., 2023. A branch-and-price-and-cut algorithm for the truck-based drone delivery routing problem with time windows. *European J. Oper. Res.* 309 (3), 1125–1144.
- Zhou, H., Qin, H., Cheng, C., Rousseau, L.-M., 2023. An exact algorithm for the two-echelon vehicle routing problem with drones. *Transp. Res. B* 168, 124–150.